

PROJECT DOCUMENTATION

Local Movers Marketplace

Connecting customers with local packers-and-movers vendors

Requirements • Functional flows • Technical specification • UI/UX preparation

Product surfaces	Proposed stack
Customer and worker mobile app	React Native
Vendor, public and admin web	Next.js
Shared business API	Express + Node.js
Operational database	MongoDB

Planning baseline | 10 September 2026

Prepared from the project brief. Business assumptions are marked as proposals. Application implementation has not started.

Document guide

This PDF combines the complete four-part project documentation into a single review copy. Start with scope and business decisions, then review workflows, technical contracts, and the UI/UX and delivery plan.

1. Project scope and business rules

Purpose, roles, MVP boundaries, service details, commercial assumptions, and decisions to confirm.

2. Functional requirements and process flows

Customer, vendor, worker, and admin requirements; booking states; payment, cancellation, delivery, and support flows.

3. Architecture, data model, and API specification

Application structure, backend modules, MongoDB collections, API inventory, security, reliability, and release controls.

4. UI/UX preparation and delivery plan

Screen inventory, error states, design handoff, development phases, acceptance tests, and pilot operations.

How to read the assumptions

The marketplace purpose, four roles, requested technology stack, and documentation-first scope come from the brief. Quote-based booking, payment collection, commission, launch city, languages, and policy windows are proposed choices that must be confirmed before becoming binding product rules.

The diagrams are rendered for PDF readability. The detailed requirements and state tables govern exception handling.

1. Project Scope and Business Rules

1.1 Understanding the project

When a tenant or homeowner shifts to another house, they need someone to pack, load, transport, unload, and sometimes unpack their belongings. Local moving businesses provide these services, but customers often have to call several vendors, repeat their requirements, compare unclear prices, and coordinate workers manually.

This project creates a marketplace between those customers and local moving vendors. The platform organizes discovery, quoting, booking, job updates, and support. It does not automatically own moving trucks or employ each vendor's workers; vendors remain responsible for delivering their accepted services. The exact contractual responsibilities must be agreed before launch.

Example: A customer moving from a rented 1BHK home submits pickup and delivery details, item information, access conditions, and a preferred date. Eligible local vendors receive the request and submit itemized quotes. The customer accepts one quote and pays any required advance. The vendor assigns a team and vehicle. Workers update the move status, and the customer confirms delivery. The platform records completion, collects feedback, and manages any issue.

1.2 Goals and success measures

Goal	Suggested measurement
Make finding movers easier	Percentage of valid requests receiving at least one quote
Improve response speed	Median time from request submission to first quote
Build customer confidence	Completed bookings, verified reviews, dispute rate
Help local vendors gain work	Active approved vendors, quote acceptance rate
Improve operational visibility	Jobs with timely assignment and status updates
Create a sustainable business	Commission earned after refunds, payment costs, and support costs

Set numerical targets after the pilot. Define each metric's denominator and time window before using it to compare performance.

1.3 Actors and responsibilities

Role	Responsibilities	Important restrictions
Customer	Manage own profile, request moves, compare quotes, book, pay, follow status, raise issues, review	Cannot see another customer's data or alter vendor pricing
Vendor owner/manager	Apply for approval, manage services, quote, assign staff/vehicles, monitor jobs, view own finances	Cannot see other vendors' quotes, workers, or finances
Worker	View assigned jobs, accept assignment, report progress, capture authorized proof, report issues	Cannot change quotes, collect arbitrary charges, or view platform finances
Admin	Review vendors, manage service areas, supervise bookings, resolve disputes, manage policy/configuration	Sensitive actions require permission and an audit trail

An account may hold multiple approved memberships, such as customer and worker. Switching roles changes the interface, but the server independently verifies permissions for every request.

1.4 Proposed MVP boundary

Launch assumption: Start in one selected city and nearby approved areas, with local residential moves. Tamil and English support should be considered during UI/UX because of the target audience; the initial language set is not yet confirmed.

Included:

- Phone-based login and account recovery process.
- Customer and worker mobile journeys.
- Vendor registration, document submission, admin approval, and suspension.
- Service area and service-package management.
- Residential moving requests with items, photos, addresses, and access details.
- Vendor eligibility matching and itemized quotes.
- Quote comparison, selection, and booking reservation.
- One selected online payment provider, if the business uses online advances.
- Booking cancellation, refund requests, and operational exception handling.
- Vendor worker/vehicle assignment and worker progress updates.
- Customer delivery confirmation, support tickets, disputes, and reviews.
- Notifications and basic administrative reporting.

Deferred unless the business explicitly expands the MVP:

- Interstate/international relocation and complex commercial moves.
- Continuous live GPS tracking and route optimization.
- Automatic or AI-generated binding prices.
- Auctions, dynamic bidding rounds, subscriptions, and promotional wallets.
- Warehousing, insurance product sales, equipment rental, and recurring contracts.
- Multi-vendor fulfillment of one booking.
- Automated marketplace payout splitting unless required by the selected payment model.
- A separate worker app binary and a full customer web booking application.

Status updates are part of MVP; they should not be presented as live location tracking.

1.5 Services and request information

The initial service catalog can offer packing, loading, transport, unloading, unpacking, and basic furniture disassembly/reassembly. Each vendor must explicitly identify which services it includes. Specialized handling is an additional agreed service, not an assumed inclusion.

A move request should collect:

- Customer contact and a preferred communication language.
- Pickup and destination addresses, map location when available, and local landmarks.
- Property type or home size, floor, lift availability, stairs, parking access, and distance from parking to the door at both ends.
- Preferred moving date and time window, with flexibility if applicable.
- Item list and approximate quantities; bulky, fragile, and high-value item declarations.

- Optional item/access photos and special handling notes.
- Packing, unpacking, labor, vehicle, and assembly requirements.
- Customer acknowledgement of the agreed excluded/prohibited item policy.

Photos can reveal private household information. Explain their purpose and share them only with eligible vendors who need them. Exact addresses and direct contact details should ordinarily be released after confirmed booking; pre-booking views show locality and necessary access information.

1.6 Commercial model - proposal to validate

The recommended planning baseline is a commission per successfully completed booking. Store the applicable commission rule with each booking so future configuration changes do not rewrite existing agreements.

Possible payment arrangements:

Model	Consequence
Platform collects full customer payment	Strong payment visibility; requires a defined vendor settlement and refund process
Platform collects an advance; balance paid directly to vendor	Lower online amount; weaker balance-payment visibility and commission reconciliation
Customer pays vendor directly; platform invoices commission	Simplest customer payment integration; greater collection and off-platform leakage risk

For process examples, assume the platform collects an advance and a remaining online balance through a provider, then settles the vendor according to an agreed policy. This is a **design assumption**, not a decision to hold funds or an assertion that any particular arrangement is permitted. Validate the provider's supported marketplace flow and contractual setup before implementing payments. Do not describe the platform as an escrow service unless that arrangement is actually established.

Illustrative calculation only: a ₹10,000 agreed service amount, a 20% advance, and a 10% commission would imply a ₹2,000 advance, ₹8,000 remaining amount, and ₹1,000 commission before tax, fees, or adjustments. These percentages are not approved business rules. Taxes, fee responsibility, rounding, and invoice ownership remain unresolved.

1.7 Quote and price rules

- Vendors submit fixed itemized quotes for the customer's current request revision.
- Each quote includes currency, service lines, quantities, discounts, applicable tax lines if configured, total, exclusions, validity, and service time window.
- Vendors can mark that a survey is required before they can issue a final quote. A survey estimate cannot be booked as a binding final price.
- Quotes must state assumptions such as lift access and item counts.
- No vendor can see a competitor's quote amount.
- Editing material request details invalidates affected quotes and requires fresh confirmation.
- Booking stores an immutable snapshot of the accepted quote and cancellation policy.
- Additional items or services require a documented change order approved by the customer before a price increase is applied. Workers cannot make unilateral price changes.
- Quote comparison must distinguish included services, total amount, timing, and exclusions; price alone is insufficient.

1.8 Vendor and worker rules

- A vendor can receive leads and submit quotes only while approved and active for the relevant service area.
- Required vendor documents are configurable after the business confirms launch-country requirements. Verification status must not imply checks that were never performed.
- Vendors invite workers; workers verify their own account and accept membership.
- A worker can access only jobs assigned through an active vendor membership. Cross-vendor sharing is never implicit.
- The MVP may restrict each worker to one active vendor membership to simplify scheduling; this requires confirmation.
- Vendors must not double-book workers or vehicles for overlapping assignment windows.
- Suspending a vendor blocks new business but creates an admin queue for existing bookings; it must not silently cancel ongoing moves.

1.9 Cancellation, failure, and dispute policy

The application needs a configurable policy with cancellation cutoffs, fees, vendor no-show handling, customer no-show handling, and refund eligibility. The business must approve the actual values before launch.

- Before quote acceptance, a customer may close an open request.
- A pending-payment reservation expires after a configurable window; late payment success goes to reconciliation, not an automatic duplicate booking.
- A confirmed booking cancellation records the actor, reason, applicable policy snapshot, fee calculation, refund decision, and notifications.
- After work begins, use an exception/dispute process instead of allowing a normal self-service cancellation that conceals performed work.
- If a vendor cannot fulfill a booking, notify the customer and admin. Another vendor requires a new customer-approved quote; never silently replace the contracted vendor.
- A dispute records evidence, participants, requested remedy, deadlines, admin decision, and financial adjustments.
- Delivery confirmation is not a waiver of damage or service complaints. A configured reporting window governs later issues.
- Refund completion requires provider confirmation. An approved refund is not yet a completed refund.

1.10 Decisions needed before UI/UX sign-off

Decision	Proposed starting point	Owner
Product name and branding	Temporary working name	Business
First city and covered radius/localities	One city pilot	Business
Languages	English and Tamil	Business
Booking method	Multiple vendor quotes, customer chooses	Business
Payment collection and settlement	Online advance and balance; provider-supported vendor settlement	Business/finance
Advance, commission, fees, and taxes	Configurable; values undecided	Business/finance
Cancellation and dispute windows	Configurable; values undecided	Operations
Verification documents and process	Admin-reviewed vendor application	Operations
Customer/worker app packaging	One codebase with separate role areas	Product
Worker membership model	One active vendor in MVP	Operations
Survey workflow	Vendor requests clarification/survey before final quote	Operations
Operational support hours and escalation	To be defined	Operations

The specification can proceed with these assumptions. Designs and implementation should revisit any decision that changes customer promises or financial behavior.

2. Functional Requirements and Process Flows

2.1 End-to-end service flow



If payment fails, retry or let the reservation expire. Confirmation requires verified payment or the approved no-advance policy.

Settlement is conditional on payment and dispute checks. Completion alone does not authorize money movement.

2.2 Customer requirements

ID	Requirement	Expected behavior
C01	Register/login	Verify phone OTP; show retry limits and expiry; preserve draft where safe
C02	Manage profile	Update name, language, own addresses and notification preferences
C03	Create request	Save a draft; validate location, future date/time, required services and access fields
C04	Submit request	Show serviceability result; create one request despite repeated taps
C05	Manage request	View matching progress; edit an unbooked request with revision tracking; close request
C06	Compare quotes	Show vendor, verified-review summary, total, inclusions, exclusions, validity and time window
C07	Accept quote	Revalidate availability and quote revision; show terms before acceptance
C08	Pay	Start/retry checkout; display backend-confirmed status and receipt/reference
C09	View booking	Show agreed scope, vendor, time, assigned team when available and event timeline
C10	Approve extra work	Review itemized change order and revised amount before approving or rejecting
C11	Cancel/reschedule	Show applicable policy and estimated financial effect; confirmed-date changes need vendor acceptance
C12	Confirm delivery	Confirm using delivery code or approved fallback; report missing/damaged items separately
C13	Get support	Open a ticket linked to request/booking; attach evidence and follow responses
C14	Review	One customer review per completed booking; edit within configured rules

Rescheduling keeps the old confirmed date until both parties accept the new schedule and any pricing change. Assignment availability must be rechecked before committing it.

2.3 Vendor requirements

ID	Requirement	Expected behavior
V01	Apply	Submit business/contact information, areas, services, and required documents
V02	Track approval	View pending, changes requested, approved, rejected or suspended status and reasons
V03	Manage offering	Configure service areas, available services, vehicle records and operational availability
V04	View eligible leads	Show authorized request details; accept interest, decline, or ask scoped clarification
V05	Quote	Draft and submit itemized final quote or request survey; show validity and assumptions
V06	Revise/withdraw quote	New revision before acceptance; accepted quote cannot be overwritten
V07	Manage bookings	View own confirmed bookings, schedules, policy snapshots and operational alerts
V08	Manage workforce	Invite workers, deactivate membership, and see invitation/acceptance status
V09	Assign team	Assign available workers and vehicle, appoint lead worker, handle rejection and reassignment
V10	Propose changes	Submit reason and itemized change order for customer approval
V11	Handle exceptions	Report delays, breakdowns, no-shows, damage and service interruption
V12	View finances	View own collected amounts, pending balances, commission, refunds and settlement statements

The vendor portal must identify overdue quotes, unassigned upcoming jobs, rejected assignments, and pending customer approvals. Vendor employees should not automatically receive owner-level finance permissions.

2.4 Worker requirements

ID	Requirement	Expected behavior
W01	Activate account	Verify identity channel and accept a legitimate vendor invitation
W02	View assignments	See own upcoming/current jobs; customer details limited to operational need
W03	Accept/reject	Confirm assignment or provide a rejection reason, notifying the vendor
W04	Execute job	Lead worker advances permitted milestones; other workers can submit notes/proof
W05	Add evidence	Upload item/checklist photos with authorization and timestamps
W06	Report issue	Record description and evidence; alert vendor and support where necessary
W07	Complete delivery	Submit proof and customer confirmation; never read a stored plaintext delivery code
W08	Work with poor connectivity	Preserve drafts and queued uploads; show unsynced state and reconcile safely

Offline progress is provisional. The server may reject a stale or out-of-order transition and must return the current authoritative state.

2.5 Admin requirements

ID	Requirement	Expected behavior
A01	Secure access	Restricted admin accounts and stronger authentication; no public admin registration
A02	Vendor review	Inspect submitted documents, request changes, approve/reject with reasons and audit history
A03	Marketplace configuration	Manage areas, services, policy versions and approved commercial settings
A04	Booking oversight	Search by reference, date, status, vendor and customer; inspect event history
A05	Customer support	Triage tickets, document actions, resolve disputes and communicate decisions
A06	Financial operations	Inspect reconciliation, approve authorized adjustments, track refunds and settlements
A07	Account moderation	Suspend access with reasons; route impacted live bookings to operations
A08	Reviews	Moderate abusive content with reason and history; do not fabricate ratings
A09	Reporting	Show marketplace activity and financial totals with clear definitions
A10	Audit	Record sensitive reads/actions, state overrides, permission changes and finance actions

Separate support, verification, and finance permissions even if a small pilot initially has one admin operator. Manual overrides must use validated commands with reasons, not unrestricted database editing.

2.6 Role access matrix

Resource/action	Customer	Vendor	Worker	Admin
Request details	Own	Eligible limited view	Assigned booking details only	Authorized operational access
Submit quote	No	Own vendor only	No	No; can moderate
Accept quote/pay	Own	No	No	Can assist through audited support, not impersonate payment
Booking details	Own	Own vendor	Assigned only	Authorized operational access
Assign workers/vehicle	No	Own vendor	No	Audited operational override
Advance job status	Confirm delivery	Allowed supervisor fallback	Assigned lead worker	Audited override
Finance	Own charges/refunds	Own vendor	No	Finance permission
Vendor approval	No	View own result	No	Verification permission

2.7 State machines

Maintain separate request, quote, booking, assignment, payment, refund, and settlement states. A single status field cannot safely describe all of them.

Moving request

DRAFT → OPEN → RESERVED → BOOKED

- **DRAFT → CLOSED** if discarded.
- **OPEN → CLOSED** if cancelled before selection.
- **OPEN → EXPIRED** after the configured request expiry.
- **RESERVED → OPEN** if payment fails/expires and the request remains valid.
- **RESERVED → BOOKED** only after booking confirmation succeeds.
- A booked request stays linked to its booking even if that booking is later cancelled. A retry with another vendor uses an explicit new/reopened request workflow with history and customer approval.

Quote

DRAFT → SUBMITTED → ACCEPTED

- **SUBMITTED → WITHDRAWN, EXPIRED, SUPERSEDED, or NOT_SELECTED.**
- Selection temporarily reserves the submitted quote via the booking reservation.
- Confirmation changes the chosen quote to **ACCEPTED** and other current quotes to **NOT_SELECTED**.
- Reservation expiry releases the quote only if still valid and unchanged; otherwise it expires.

Booking/service

Current state	Next state	Authorized actor and condition
PENDING_PAYMENT	CONFIRMED	System; verified required payment or configured no-advance policy
PENDING_PAYMENT	EXPIRED	System; reservation timeout, reconciled against payment events
CONFIRMED	ASSIGNED	Vendor; valid team and vehicle attached
ASSIGNED	EN_ROUTE_PICKUP	Lead worker; accepted assignment and service window
EN_ROUTE_PICKUP	ARRIVED_PICKUP	Lead worker; arrival recorded
ARRIVED_PICKUP	PACKING	Lead worker; packing included
ARRIVED_PICKUP/PACKING	LOADING	Lead worker; required packing complete or legitimately omitted
LOADING	IN_TRANSIT	Lead worker; loading checklist complete
IN_TRANSIT	ARRIVED_DROPOFF	Lead worker
ARRIVED_DROPOFF	UNLOADING	Lead worker
UNLOADING	AWAITING_CONFIRMATION	Lead worker; included unpacking/assembly checklist and proof complete
AWAITING_CONFIRMATION	COMPLETED	Customer confirmation; admin fallback requires documented evidence
CONFIRMED/ASSIGNED/EN_ROUTE_PICKUP	CANCELLED	Authorized cancellation policy evaluation

PENDING_PAYMENT can also become **CANCELLED** by customer action. After arrival/work starts, admin-managed service termination may move the booking to **TERMINATED** with recorded performed work and a financial decision. Do not mark incomplete service as completed.

An issue/dispute is a linked record and may coexist with a booking state. A delay does not automatically reset the service timeline. Reassignment invalidates old worker permissions; if it leaves no accepted team before departure, return **ASSIGNED** to **CONFIRMED** through an audited reassignment command.

Assignment

PROPOSED → **ACCEPTED** or **REJECTED**; an active assignment may become **REPLACED** or **CANCELLED**. Booking assignment status and individual acceptance must be visible separately.

Finance

- Payment attempt: **CREATED** → **PENDING** → **SUCCEEDED** | **FAILED** | **EXPIRED**.
- A late provider success can correct an expired attempt through reconciliation; whether to confirm a booking or refund depends on reservation ownership.

- Refund: REQUESTED → APPROVED | REJECTED; approved refunds become PROCESSING → SUCCEEDED | FAILED.
- Settlement: NOT_ELIGIBLE → ON_HOLD | ELIGIBLE → PROCESSING → PAID | FAILED.
- Partial/full refunded totals are derived from successful refund records. Do not overwrite a successful payment with a generic “refunded” flag and lose its history.

2.8 Detailed workflows and exceptions

Vendor onboarding

1. Vendor registers and submits an application.
2. Server validates required data and stores documents privately.
3. Admin reviews and approves, rejects, or requests corrections with reasons.
4. Vendor receives a notification and can inspect the decision.
5. Approved vendor configures services and workforce and becomes eligible for matching.
6. Material identity/document changes create a new review where required by policy.

Request matching and no quotes

1. Validate pickup/destination serviceability and requested date.
2. Select approved vendors whose service areas and supported services fit the move.
3. Issue lead invitations to a configurable number of eligible vendors; record delivery status.
4. Keep the request OPEN while waiting, show a realistic response deadline, and notify on new quotes.
5. If no vendors or quotes are available, show that explicitly. Offer editing date/details or support. Do not create a booking or fabricate estimates.

Quote acceptance and payment

1. Customer chooses a valid final quote and reviews its full scope and policy.
2. Backend atomically checks request revision, vendor eligibility, quote validity, and absence of another active booking reservation.
3. Create a booking reservation and payment order using an idempotency key.
4. Customer completes provider checkout. App redirect alone is insufficient evidence of success.
5. Backend verifies provider webhook/status and atomically confirms the reservation.
6. Notify customer/vendor, close competing quotes, and create an assignment task.
7. If payment succeeds after expiry, reconcile availability and customer authorization. If confirmation is unsafe, refund/review the payment; never allocate two bookings.

Move execution and delivery

1. Vendor assigns workers and vehicle; workers accept.
2. Lead worker sees permitted contact/address data and travels to pickup.
3. Record arrival, packing if included, loading checklist, transit, arrival at destination, and unloading.
4. For extra scope, pause the affected additional work and request customer-approved change order.
5. Submit delivery proof and request customer confirmation.
6. Customer confirms using a short-lived code or in-app confirmation.
7. If the customer is unavailable, support reviews evidence under an approved fallback process. No automatic completion merely because time elapsed in the proposed MVP.
8. Record completion and initiate balance/settlement checks independently.

Refund and settlement

1. Evaluate cancellation/dispute against the booked policy and performed work.
2. Authorized operator or deterministic policy approves a specific refund amount.
3. Server prevents refunding more than the available refundable amount, including in-flight refunds.
4. Submit refund through provider and await confirmed outcome.
5. Settlement eligibility checks successful collections, approved adjustments, commission, dispute holds, and previous settlements.
6. Record payout reference and confirm outcome through the supported provider or an audited manual bank-transfer reconciliation process.
7. A dispute after settlement follows a documented recovery/adjustment process; never silently edit a paid settlement.

2.9 Notifications

Event	Recipients	Channels
Login challenge	Account owner	OTP channel
Vendor approval/correction	Vendor	Portal inbox; optional email/push
New eligible request	Vendor	Portal inbox; selected alert channel
Quote submitted/expiring	Customer	In-app and push
Booking/payment confirmed	Customer and vendor	In-app; receipt through selected channel
Assignment/new schedule	Assigned worker	In-app and push
Assignment rejected	Vendor	Portal inbox and alert
Job milestones/delay	Customer and vendor	In-app and selected push events
Change order	Customer	In-app and push
Cancellation/refund/dispute decision	Relevant participants	In-app and selected transactional channel
Unassigned job/provider failure	Authorized operations team	Admin alert

Notifications are retried asynchronously and deduplicated. The product timeline remains available if push/SMS fails. Lock-screen messages should not expose full addresses or sensitive evidence.

3. Architecture, Data Model, and API Specification

This document specifies the intended implementation. It is not application code. Package versions, hosting vendors, and provider SDKs will be selected and verified during development.

3.1 Proposed architecture

React Native app Next.js vendor/public web Next.js admin portal
Shared Express API on Node.js
MongoDB Private object storage Payment provider
Durable job queue + background workers
OTP / notifications Payment webhooks + reconciliation

Next.js provides React-based web interfaces. React Native provides mobile interfaces. Express is the authoritative business API running on Node.js. MongoDB stores operational records. File content belongs in private object storage, not in ordinary MongoDB booking documents.

Use a modular monolith first: one backend codebase with clearly separated modules and an independently runnable background worker. Microservices are unnecessary for the initial scope.

Recommended implementation language: TypeScript across web, mobile, and server. Share contracts and validation definitions where practical, while keeping backend authorization authoritative.

3.2 Proposed repository structure

apps/	
web/	# Public website and vendor portal, Next.js
admin/	# Restricted admin interface, Next.js
mobile/	# Customer and worker navigation, React Native
api/	# Express HTTP API and business modules
jobs/	# Queue processing and scheduled reconciliation
packages/	
contracts/	# API types, enums, shared validation contracts
config/	# Shared lint/TypeScript configuration
domain/	# Server-only domain logic shared by API and jobs
docs/	# Requirements, architecture, API and decision records
infra/	# Deployment definitions when hosting is chosen

Do not place server secrets or server-only modules in shared client packages. Using separate admin deployment helps restrict access but does not replace authorization.

3.3 Backend modules

Module	Responsibility
Authentication	OTP, sessions, token rotation, logout and recovery
Accounts and memberships	Profiles, roles, vendor membership and invitations
Vendor verification	Applications, document review and approval lifecycle
Catalog and serviceability	Areas, services, eligibility rules
Moving requests	Drafts, revisions, inventory and lead invitations
Quotes	Itemized prices, revisions, expiry and acceptance validation
Bookings	Reservations, policy snapshots and lifecycle commands
Dispatch	Workers, vehicles, assignments and availability checks
Job execution	Milestones, checklists, delivery confirmation and evidence
Finance	Payment attempts, refunds, commission and settlement ledger
Support	Tickets, disputes, decisions and escalation
Communication	Notification inbox, device tokens and asynchronous delivery
Administration	Configuration, audit access and reports

3.4 Data model

All primary records use a stable ID, [createdAt](#), and [updatedAt](#). Use UTC timestamps and store the service time zone separately for customer-facing scheduling. Mutable workflow records carry a [version](#) for concurrency checks. Store money as integer minor units with an explicit currency; never use floating-point arithmetic for financial amounts.

Collection	Main fields and relationships
users	normalized phone, displayName, language, accountStatus, verifiedAt
sessions	userId, hashed refresh token/session reference, expiry, revokedAt, device metadata
otpChallenges	destination reference, hashed code, expiry, attempt count, consumedAt
vendorMemberships	vendorId, userId, role, status, invitation/acceptance metadata
vendorApplications	applicantId, business details, document file IDs, review state and reasons
vendors	ownerId, businessName, status, serviceAreaIds, serviceIds, verification summary
serviceAreas	locality definitions or supported geographic geometry, status, timeZone
services	service code, label, description, supported checklist configuration
addresses	userId, label, address fields, coordinates; booking/request uses a snapshot

Collection	Main fields and relationships
movingRequests	customerId, pickup/destination snapshots, schedule window, services, item list, file IDs, revision, status
leadInvitations	requestId, vendorId, requestRevision, response state, delivery timestamps
quotes	requestId, vendorId, requestRevision, quoteRevision, itemized lines, currency, total, assumptions, expiry, status
bookings	requestId, customerId, vendorId, quoteId, immutable quote/policy snapshots, schedule, state, version, reservationExpiry
vehicles	vendorId, identifier, type, capacity description, availability/status
assignments	bookingId, vendorId, workerUserIds, leadWorkerId, vehicleId, reserved interval, acceptance states, version
bookingEvents	bookingId, sequence/version, actorId, old/new state, eventType, timestamp, evidence references
changeOrders	bookingId, proposedBy, lines, priceDelta, schedule impact, status, customer decision and timestamp
paymentAttempts	bookingId, purpose, amount, currency, provider/order/payment IDs, idempotency reference, status
refunds	bookingId, paymentAttemptId, amount, reason, approval metadata, providerRefundId, status
ledgerEntries	bookingId, vendorId, entryType, signed amount, currency, source record, posting group, timestamp
settlements	vendorId, included ledger references, amount, currency, status, payout reference
supportTickets	openedBy, bookingId/requestId, category, messages/evidence, assignee, state
disputes	bookingId, ticketId, requested remedy, evidence, decision, settlement hold, resolution metadata
reviews	bookingId, customerId, vendorId, rating, text, moderation status
files	ownerId, vendorId/bookingId scope, objectKey, contentType, size, scan status, purpose
notifications	recipientId, eventId, template, safe payload, channel status, readAt
deviceTokens	userId, deviceId, push token, active status
policyVersions	policy type, immutable version, effective date, configured values
auditLogs	actorId, permission context, target, action, safe before/after fields, reason, requestId
webhookEvents	provider, eventId, verified timestamp, processing status, safe payload reference
outboxEvents	transaction-linked event, processing state, retry count, nextAttemptAt
idempotencyRecords	actor/scope, key, request fingerprint, result reference, expiry

The precise schema can split large item lists, messages, or event histories into child collections. Avoid unbounded arrays in frequently updated documents.

Key relationships

- Customer → many moving requests.

- Moving request → many vendor quotes; at most one active reservation/booking at a time.
- Confirmed booking → one selected vendor and immutable accepted scope.
- Vendor → many memberships, vehicles, quotes, and bookings.
- Booking → assignment history, events, change orders, payment attempts, refunds, and support records.
- Completed booking → at most one customer review.

Indexes and constraints

- Unique normalized phone on active account identities under the chosen account-reuse policy.
- Unique vendor/user membership pair; conditional rule if only one active worker membership is allowed.
- Unique quote revision for (requestId, vendorId, quoteRevision).
- Atomic request reservation field plus a unique partial active-booking constraint on requestId.
- Unique provider payment ID, refund ID, and (provider, eventId) webhook identifier.
- Unique review bookingId and idempotency (actor/scope, key).
- Compound query indexes for customer bookings, vendor schedule, assigned-worker jobs, open leads, and support queues.
- Geospatial index only where actual matching uses stored valid geometry.
- Expiry indexes may clean OTP/session data. Business reservation/quote expiry still requires explicit checks and a job; cleanup timing is not a booking correctness mechanism.

Interval conflicts cannot be prevented by a simple unique index. Assignment writes need a concurrency-safe reservation strategy, such as locked resource calendars or transactional slot reservations, including travel/setup buffers. Validate it under concurrent tests.

3.5 API conventions

- Base path: /api/v1.
- Authenticated identity comes from the verified session/token, never a client-submitted customerId or vendorId alone.
- Resource authorization checks role, membership, tenant ownership, assignment, and current state.
- Lists use cursor pagination and server-enforced limits.
- Mutating commands return the updated resource/version or an explicit async operation reference.
- Use 400 for malformed input, 401 for missing/invalid authentication, 403 for forbidden actions, 404 where appropriate to avoid resource enumeration, 409 for state/version conflicts, 422 for semantic validation, and 429 for limits.
- Error contract: { error: { code, message, fieldErrors? }, requestId }.
- Require idempotency keys for booking creation, payment/refund requests, and other retry-sensitive commands.
- For version-sensitive changes send expectedVersion; reject stale writes with the current safe resource state.
- Publish an OpenAPI specification during implementation and use it for client contract validation.

3.6 Proposed API inventory

Names are a starting contract; implementation should finalize payloads and permissions before client development.

Group	Endpoints
Auth	POST /auth/otp/request, POST /auth/otp/verify, POST /auth/refresh, POST /auth/logout
Account	GET /me, PATCH /me, GET/POST /me/addresses, PATCH/DELETE /me/addresses/:id, POST /me/device-tokens

Group	Endpoints
Catalog	GET /services, GET /service-areas, POST /serviceability/check
Vendors	POST /vendor-applications, GET/PATCH /vendor-applications/:id, POST /vendor-applications/:id/submit, GET/PATCH /vendor/profile
Workforce	GET /vendor/workers, POST /vendor/worker-invitations, POST /worker-invitations/:id/accept, PATCH /vendor/workers/:id/membership
Vehicles	GET/POST /vendor/vehicles, PATCH /vendor/vehicles/:id
Requests	POST /moving-requests, GET /moving-requests, GET/PATCH /moving-requests/:id, POST /moving-requests/:id/submit, POST /moving-requests/:id/close
Vendor leads	GET /vendor/leads, POST /vendor/leads/:id/decline, POST /vendor/leads/:id/clarifications, POST /vendor/leads/:id/survey-requests
Quotes	POST /moving-requests/:id/quotes, GET /moving-requests/:id/quotes, PATCH /quotes/:id, POST /quotes/:id/submit, POST /quotes/:id/withdraw
Booking	POST /bookings with quote ID/version, GET /bookings, GET /bookings/:id, GET /bookings/:id/events
Changes	POST /bookings/:id/cancellation-preview, POST /bookings/:id/cancel, POST /bookings/:id/reschedule-requests, POST /reschedule-requests/:id/decision
Dispatch	POST /bookings/:id/assignments, POST /assignments/:id/accept, POST /assignments/:id/reject, GET /worker/jobs
Execution	POST /bookings/:id/transitions, POST /bookings/:id/evidence, POST /bookings/:id/issues
Completion	POST /bookings/:id/delivery-challenge, POST /bookings/:id/confirm-delivery
Change orders	POST /bookings/:id/change-orders, POST /change-orders/:id/approve, POST /change-orders/:id/reject
Payments	POST /bookings/:id/payment-orders, GET /bookings/:id/payments, POST /webhooks/payments/:provider
Support	POST /support-tickets, GET /support-tickets, GET /support-tickets/:id, POST /support-tickets/:id/messages
Reviews	POST /bookings/:id/review, GET /vendors/:id/reviews
Files	POST /files/upload-intents, POST /files/:id/finalize, GET /files/:id/access
Notifications	GET /notifications, POST /notifications/:id/read
Admin verification	GET /admin/vendor-applications, POST /admin/vendor-applications/:id/decision
Admin operations	GET /admin/bookings, POST /admin/bookings/:id/override, POST /admin/vendors/:id/suspend
Admin finance	POST /admin/refunds, GET /admin/refunds, GET /admin/settlements, POST /admin/settlements/:id/process
Admin support/reporting	POST /admin/disputes/:id/decision, GET /admin/reports, GET /admin/audit-logs
Admin configuration	GET/POST /admin/policies, GET/POST /admin/service-areas, PATCH /admin/service-areas/:id

Shared endpoints return role-appropriate projections. A generic `/bookings/:id/transitions` endpoint still accepts only enumerated commands with actor/state checks; it is not arbitrary status editing.

3.7 Reliability and consistency

- Use MongoDB transactions where booking/request/quote/payment-linked writes must commit together; deployment must support those transactions.
- Atomically claim the request before creating its active booking. A concurrency test must prove that two simultaneous acceptances cannot both win.
- Store committed business events in an outbox in the same transaction. A worker delivers notifications/jobs and records deduplication IDs.
- Payment calls are external and cannot join a database transaction. Persist intent, use provider idempotency where supported, process verified callbacks, and reconcile incomplete attempts.
- Verify webhook authenticity against the provider's documented mechanism and deduplicate before financial side effects.
- Accept repeated events safely and handle out-of-order events without reverting confirmed success to pending/failed.
- Use bounded retries with backoff, dead-letter handling, and operational alerts for exhausted work.
- Do not use process memory or a browser timer as the only source of reservation expiry or payment status.
- Keep service completion, customer receivable, refund liability, commission, and vendor payable distinct in the ledger. Reversals are new entries, not edits to history.

3.8 Security and privacy requirements

- Enforce object-level authorization for every ID-based operation and tenant-scoped database query.
- Limit OTP sends and verification attempts by destination, device/session, and network signals; never log OTPs.
- Use secure web session cookies with appropriate CSRF protection; protect mobile refresh credentials with platform secure storage. Final auth design must document token rotation and revocation.
- Require stronger admin authentication and permission checks for refunds, settlements, role grants, and overrides.
- Keep payment card details out of the application; use provider-hosted/tokenized flows.
- Give uploads size/type limits, scan/quarantine handling, private storage, and short-lived authorized access URLs.
- Strip unnecessary image metadata where appropriate and avoid public document URLs.
- Never put secrets, access tokens, full addresses, or raw verification documents in analytics or routine logs.
- Record consent/notice versions, retention settings, deletion requests, and export/support processes. Finance/audit retention and account deletion rules require jurisdiction-specific review before launch.
- Restrict workers' access after assignment removal and after the operational access period ends.
- Apply request validation, input limits, safe database query construction, and endpoint-specific rate limits.

3.9 Proposed nonfunctional targets

These are initial engineering targets to validate through pilot capacity planning, not production guarantees.

Area	Initial target/requirement
API responsiveness	p95 below 700 ms for ordinary reads/writes under agreed load, excluding uploads/external checkout
Availability	Aim for 99.5% monthly pilot service availability; measure before promising an SLA
Consistency	No duplicate active bookings, unauthorized tenant access, or duplicate financial side effects

Area	Initial target/requirement
Accessibility	Screen-reader labels, usable focus/navigation, sufficient contrast and scalable text
Connectivity	Clear retries and unsynced state for mobile; no false successful payment/progress screens
Observability	Structured request IDs, error monitoring, queue lag, payment reconciliation alerts and audit trail
Recovery	Encrypted backups and tested restore; proposed RPO 24 hours/RTO 8 hours need business approval
Financial recovery	Reconcile provider transaction history after restore; the database backup alone is insufficient

3.10 Environments and release controls

Use isolated development, staging, and production environments with distinct databases, storage, credentials, and payment modes. Never copy unredacted production customer records into developer fixtures.

CI should validate formatting/types, focused business tests, API contracts, and production builds. Staging must test provider callbacks and complete four-role journeys. Maintain rollback instructions, schema migration/versioning scripts, secrets management, and a restore runbook. Hosting choices and operational ownership must be confirmed before deployment.

4. UI/UX Preparation, Delivery Plan, and Acceptance Criteria

4.1 UI/UX objective

The next phase should make a household move understandable: what information to provide, what each vendor includes, what the customer will pay, who will arrive, what happens next, and how to resolve a problem.

Build designs around the workflows and permissions already documented. Do not start with isolated attractive dashboard screens that cannot complete a booking.

4.2 Screen inventory

Customer mobile app

1. Welcome, language selection, phone login, OTP, and profile setup.
2. Home: create a move, active booking card, and past moves.
3. Request wizard: locations → date/home/access details → inventory/photos → services → review/submit.
4. Matching progress, no vendors available, and request edit/close.
5. Quote list, comparison, vendor profile/reviews, and full quote detail.
6. Quote acceptance summary, cancellation terms, payment checkout, pending/success/failure.
7. Booking detail: scope, timeline, schedule, vendor/team and support action.
8. Change-order review and schedule-change review.
9. Cancellation preview, cancellation outcome, and refund tracking.
10. Delivery confirmation, issue reporting, completion and review.
11. Booking history, charges/receipts, notification inbox, saved addresses, support tickets and settings.

Suggested navigation: Home, My Moves, Notifications, Profile. Place support contextually within each booking and in Profile.

Vendor web portal

1. Login, application wizard, document upload, review status and corrections.
2. Dashboard with upcoming jobs, pending leads, assignment alerts and financial summary.
3. Leads list/detail, clarification/survey request, quote builder and quote history.
4. Bookings list/calendar and booking detail.
5. Worker invitations, worker list and membership status.
6. Vehicles and availability.
7. Team/vehicle assignment and acceptance status.
8. Job timeline, incident reporting and change-order creation.
9. Transactions, commission and settlement statements.
10. Business profile, services, service areas, notification settings and support.

Worker mobile area

1. Login, invitation acceptance and membership status.
2. Today's jobs and upcoming jobs.
3. Assignment detail and accept/reject.

- 4. Active job with one primary next-step action.
- 5. Item/service checklist, photo capture and upload status.
- 6. Issue reporting and contact options.
- 7. Delivery confirmation flow, completed job summary and own work history.

Worker screens should support one-handed use, large touch targets, clear local-language status labels, and obvious offline/pending indicators.

Admin web portal

- 1. Secure login and operational overview.
- 2. Vendor approval queue, document review and decision history.
- 3. Customers, vendors, workers and account restrictions.
- 4. Bookings, timeline, exception queue and audited action dialogs.
- 5. Tickets/disputes, evidence and resolution.
- 6. Payment reconciliation, refunds and settlement operations.
- 7. Service areas, catalog, policy versions and commercial configuration.
- 8. Reports, review moderation, admin permissions and audit log.

Public website

Home, how it works, covered areas, service descriptions, vendor application entry, contact/support, and approved policy pages. Do not display unsupported coverage, fabricated vendor counts, or placeholder reviews as real claims.

4.3 Required UI states

For every relevant screen, specify loading, empty, validation error, permission denied, retry, and success behavior. Critical additional states include:

Screen	States that must be designed
OTP	Expired code, rate limit, wrong code, lost access/recovery
Request	Unsupported area, invalid date, incomplete draft, failed photo upload
Quotes	No quotes, expired quote, revised request, vendor suspended, reservation already taken
Checkout	Pending verification, failed payment, app closed, duplicate tap, late success under review
Assignment	No available team, worker rejected, vehicle conflict, reassignment
Job	Delay, offline draft, rejected stale update, incident, customer unreachable
Completion	Incorrect/expired code, missing proof, disputed delivery, admin fallback pending
Finance	Partial refund, failed refund, held settlement, unpaid balance

The primary action label must describe the actual commitment: “Submit request,” “Accept quote,” or “Pay advance,” rather than reusing “Confirm” everywhere.

4.4 UI/UX deliverables and order

- 1. Validate launch assumptions, personas, payment model, and cancellation rules.
- 2. Create sitemap and four-role user journey maps.

3. Wireframe the customer request-to-completion path and vendor quote-to-assignment path together.
4. Wireframe worker execution and admin exception handling.
5. Review content fields, empty/error states, and Tamil/English text expansion.
6. Establish typography, color, spacing, accessible controls, forms, cards and timeline components.
7. Produce high-fidelity screens and a clickable cross-role booking prototype.
8. Test with representative customers, local vendors, and workers; record friction and revise.
9. Hand off annotated states, field validation, API dependencies and assets.

Design sign-off criterion: A reviewer can complete a realistic moving request, select a quote, understand payment, assign a team, simulate the move, confirm delivery, and handle one cancellation or dispute without inventing missing screens.

4.5 Development phases and exit criteria

Do not assign a delivery date until team size, design completion, integrations, and approved scope are known. These are sequenced work packages rather than a fixed schedule.

Phase	Work	Exit criterion
0. Discovery	Confirm business assumptions, service areas, policies, roles and integrations	Approved scope and recorded unresolved decisions
1. UI/UX	Journeys, wireframes, visual system and prototype	Four-role prototype covers main flow and critical exceptions
2. Foundation	Repository, auth, permissions, CI, environments, contract definitions	Each role accesses only its authorized area; baseline builds pass
3. Supply onboarding	Vendor application/review, areas, services, workers and vehicles	Approved vendor can become serviceable and invite workers
4. Marketplace	Customer request, matching, quote revision/comparison	Customer receives and compares valid local quotes
5. Booking/finance	Reservation, provider integration, cancellation/refunds, ledger	Repeated/concurrent operations remain correct in sandbox tests
6. Fulfillment	Assignment, milestones, proof, change orders and completion	Team completes a staged job with correct customer timeline
7. Operations	Support, disputes, reviews, reconciliation, settlements, reports	Admin resolves realistic exceptions with traceable records
8. Pilot readiness	Accessibility, performance, security checks, backups and training	Pilot checklist passes and named operators own alerts/support
9. Pilot	Limited geography and vendor group; monitor and fix	Evidence supports expansion or identifies needed changes

4.6 Testable acceptance criteria

ID	Scenario	Pass condition
AC01	Unapproved vendor requests leads	Access denied; no customer lead data disclosed
AC02	Vendor A guesses Vendor B's booking ID	Access denied or safely not found; event audited where appropriate
AC03	Customer submits a valid local request	One open request created; eligible vendors receive deduplicated invitations
AC04	No eligible vendors	Customer sees clear no-availability state; no false quote/booking is created
AC05	Customer changes inventory after quotes	Prior revision quotes cannot be accepted; vendors can quote current revision
AC06	Two quote acceptances race	Exactly one active reservation succeeds; loser gets a conflict
AC07	Customer repeats checkout request	Same logical payment order/result is returned, with no duplicate charge intent
AC08	Payment webhook repeats or arrives out of order	One financial effect and one confirmation; state does not regress
AC09	Payment succeeds after reservation expires	Reconciliation follows explicit policy; no second booking allocation
AC10	Vendor assigns overlapping vehicle/worker jobs concurrently	At most one conflicting reservation succeeds
AC11	Removed worker tries to read or update old job	Server rejects access even if app cached an old role/job screen
AC12	Worker skips loading and reports transit	Invalid transition rejected with authoritative state
AC13	Worker is offline	Draft clearly marked unsynced; reconnect does not duplicate events or silently overwrite state
AC14	Extra service is proposed	Agreed price remains unchanged until customer approval; accepted delta is recorded
AC15	Customer cancels confirmed job	Policy snapshot determines fee/refund; all parties see correct status
AC16	Refund is approved but provider fails	Refund remains failed/pending retry; UI does not claim money returned
AC17	Customer confirms delivery	Required proof/confirmation is recorded and service completes once
AC18	Worker tries own-job confirmation without customer proof	Rejected; documented admin fallback remains separately authorized
AC19	Booking has dispute or unpaid balance	Settlement cannot proceed contrary to configured eligibility policy
AC20	Customer reviews another person's booking	Rejected; one review maximum per eligible booking

ID	Scenario	Pass condition
AC21	Private file URL requested by unrelated user	Access denied; direct storage objects are not public
AC22	Admin changes commission config	Existing booking snapshot remains unchanged; new rule is versioned
AC23	Notification service is unavailable	Booking still commits; notifications retry with visible operational failure tracking
AC24	Backup restored in test environment	Documented recovery succeeds and provider financial reconciliation is demonstrated

Use unit tests for pricing/policy/state rules, integration tests for authorization and concurrent database operations, provider sandbox tests for finance, and end-to-end tests for the four-role booking journey. Manually test mobile permissions, accessibility, poor connectivity, and actual worker usability.

4.7 Pilot operations checklist

- Named owner for vendor verification, customer support, finance reconciliation and production alerts.
- Approved service catalog, coverage, cancellation terms, dispute rules and payment setup.
- Real onboarding instructions and a small verified vendor group with trained workers.
- Customer-facing support contacts and operational hours.
- Tested payment, refund, cancellation, provider failure and vendor no-show paths.
- Tested backups, restore, rollback, access revocation and sensitive log filtering.
- Clear policy for worker evidence capture and customer document/photo retention.
- Daily pilot review of unquoted requests, unassigned bookings, delayed jobs, pending payments and unresolved disputes.

4.8 Risks and responses

Risk	Practical response
Too few vendors	Launch in a small area with vendors onboarded before customer promotion
Quote disputes	Structured inventory, access fields, explicit inclusions and approved change orders
No-show or resource conflict	Assignment deadlines, acceptance tracking and an operations exception queue
Off-platform booking	Convenient payments, documented scope, support and reliable vendor value; measure leakage
Damage or loss	Evidence workflow, clear reporting process and approved responsibility terms
Poor worker connectivity	Lightweight screens, queued drafts, visible synchronization and server validation
Finance mismatch	Immutable transaction history, idempotency and recurring reconciliation
Scope growth	Preserve the MVP boundary and record approved changes with design/API impact

4.9 Project documentation acceptance

This documentation stage is complete when the team has a shared understanding of the roles, scope, normal and exception flows, state transitions, architecture, data ownership, and next-phase deliverables. Business decisions marked as proposals still require confirmation before being turned into binding product rules.

The next authorized stage can be UI/UX design. Application code should follow approved journeys and API contracts, rather than being presented as complete before these choices are resolved.